

JULY

Meeting with Company A

AUGUST 8, 2021 - 16H

JUNE 15, 2021 - 15H

// AUGUST 2023

APRIL 15, 2021

Meeting with Company B

ARQUITECTURA Y SISTEMAS OPERATIVOS

TEACHER

MARCO USTARROZ

JUNE 15, 2021 - 15H

MARCH 22, 2021 - 15H

PROCESOS E HILOS



- Un proceso es una abstracción de un programa en ejecución, incluyendo los valores actuales del contador de programa, los registros y las variables.
- La CPU conmuta de manera muy rápida entre proceso y proceso (a esto llamamos *multiprogramación*) generando la sensación de que todos ellos se están ejecutando de manera paralela.
- Ver analogía de Cocinero pág. 85 libro de Tanembaun.

Creación y Tipos de procesos



- Hay 4 eventos principales que provocan la creación de procesos:
 - **Arranque de sistema**
 - La ejecución, desde un proceso, a una **llamada de sistema** para la creación de un proceso.
 - Una **petición de usuario para la creación de un proceso**
 - El **inicio de un trabajo por lotes**
- Distinguimos entre dos tipos de procesos:
 - Proceso en **primer plano**: que requieren la interacción del usuario
 - Procesos en **segundo plano**: aka “daemons” que se ejecutan sin necesidad de interacción, por ejemplo un demonio que espera llegadas de mail, un web dispatcher, etc.

Terminación de un proceso



- Una vez creado un proceso, éste finaliza en algún momento, y ésta finalización puede deberse a varios motivos:
 - Salida normal (termina la ejecución)
 - Salida por error (ej. invoco a gcc con un archivo que no existe)
 - Error fatal (ej. División por cero)
 - Eliminado por otro proceso (otro proceso ejecuta un kill)

Jerarquía de procesos



- En algunos sistemas, cuando un proceso crea otro, el proceso padre y el proceso hijo continúan asociados formando una jerarquía de procesos, como ocurre en UNIX
- En el caso de Windows se diferente, no existe una “jerarquía de procesos”, si no que el proceso padre al crear un hijo recibe un token que puede utilizar para manejar al hijo. Este token se lo puede pasar a otros procesos, con lo cual invalida la jerarquía.

Estados de un proceso



- Aunque los procesos son independientes, con su propio contador de programa y estado interno (código, stack, etc) a veces necesitan comunicarse entre sí.
- Por ejemplo un proceso puede necesitar de la salida de otro proceso para poder ejecutarse, como es el caso de los sgtes. Comandos de shell:

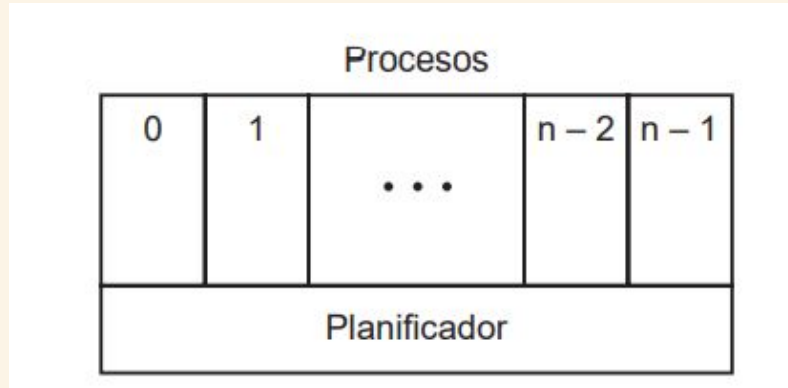
cat arch1 arch2 arch3 | grep hola

- En el ejemplo anterior cat concatena 3 archivos y la salida se la envía a grep, que mostrará las líneas que contengan la palabra “hola”.
- Dependiendo de cómo el procesador ejecute los procesos, puede ser que se ejecute primero “grep” que “cat”. En tal caso grep se quedará “bloqueado” esperando la salida de resultados de cat.

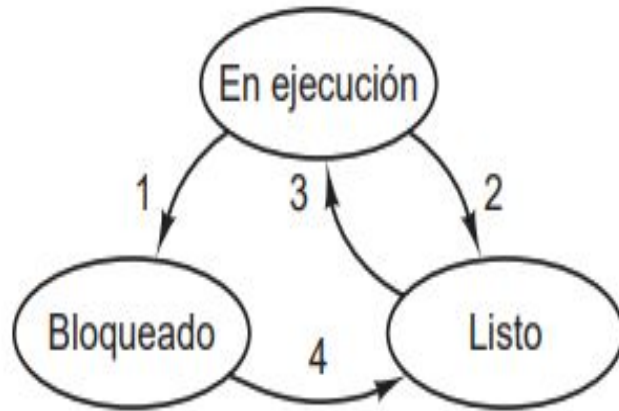
Tabla de procesos - Planificador



El sistema operativo mantiene una tabla, llamada “**Tabla de procesos**”, ésta tabla contiene información importante del estado de cada proceso, p.ej: contador del programa, asignación de memoria, recursos abiertos (archivos, impresoras..), etc.



Estados de un proceso



1. El proceso se bloquea para recibir entrada
2. El planificador selecciona otro proceso
3. El planificador selecciona este proceso
4. La entrada ya está disponible

CREANDO PROCESOS EN UNIX-LIKE SO *



Para crear procesos en sistemas operativos “UNIX-Like”, utilizamos la llamada a sistema “fork” .

“Fork” crea un nuevo proceso idéntico al proceso que lo creó (padre), ambos el proceso padre y el proceso hijo continúan la ejecución a partir de la instrucción fork.

La instrucción **fork()** devuelve **CERO** para el proceso nuevo, y el **pid (process id)** del hijo al proceso padre.

Ejemplo



```
int main( ){

    pid_t child_pid;
    child_pid = fork (); // Create nuevo proceso hijo
    if (child_pid < 0) {
        printf("FALLÓ EL FORK !");
        return 1;
    } else if (child_pid == 0) {
        printf ("ME ACABAN DE CREAR, SOY UN PROCESO HIJO !, MI PROCESS ID ES = %d, Y EL DEL MI PADRE ES = %d\n",
            getpid(), getppid( ) );
    } else {
        wait(NULL); //bloqueo al padre hasta que todos los hijos finalicen
        printf ("SOY EL PROCESO PADRE! ");
        printf ("MI PROCESS ID ES = %d, Y EL DE MI PADRE = %d \n,Y EL DEL HIJO RECIEN CREADO = %d \n",
            getpid( ), getppid( ), child_pid);
    }

    sleep(10);
    return 0;
}
```



THANKS!

CREDITS: This presentation template was created by **Slidesgo**,
including icons by **Flaticon**, infographics & images by **Freepik**